



INFORME DE PRÁCTICA

I. PORTADA

Tema:	APE 1-Lenguajes de Programación
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Ashanga Yumbo Brishy Anahy
Asignatura:	Algoritmos y lógica de programación
Docente:	Ing. José Ruben Caizabuano, Mg.

II. INFORME DE PRÁCTICA

2.1 Objetivos

General:

Conocer los lenguajes de programación de la actualidad y sus antecesores.

Específicos:

- Identificar los principales hitos en la evolución de los lenguajes de programación.
- Analizar las características de los lenguajes más utilizados en la actualidad.
- Elaborar una infografía que sintetice la información investigada.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 2

No Presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- Realice una consulta sobre LA HISTORIA Y EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN.
- Seleccionar una herramienta que permita realizar infografías.
- Realizar una infografía a manera de resumen de la lectura realizada.
- Subir el documento al aula virtual en el formato de Guías APE en PDF.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC.
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas (fuentes confiables)

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): Plataforma web y aplicación de diseño gráfico en línea (Canva).

2.6 Actividades por desarrollar



Detalladas en la guía proporcionada por el docente.

2.7 Resultados obtenidos

Se identificaron los principales hitos en la evolución de los lenguajes de programación y se elaboró una infografía que resume la relación entre lenguajes pioneros y contemporáneos. Además, se comprendió cómo los aportes de los lenguajes antecesores influyen en los actuales.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

La evolución de los lenguajes de programación refleja el avance tecnológico y las necesidades cambiantes de los desarrolladores. Desde los primeros lenguajes orientados al cálculo y la gestión empresarial hasta los actuales que priorizan la legibilidad, la seguridad y la eficiencia, cada etapa ha marcado un salto en la forma de pensar y construir software. Los lenguajes contemporáneos se dividen entre aquellos que buscan accesibilidad y rapidez (Python, Java, JavaScript) y los que se enfocan en rendimiento y confiabilidad (C++, Go, Rust), mostrando que la programación moderna combina simplicidad con potencia técnica.

2.10 Recomendaciones

Dado que la práctica cumplió satisfactoriamente con los objetivos planteados y se alcanzaron los resultados esperados, no se consideran necesarias recomendaciones adicionales.

2.11 Referencias bibliográficas

- [1] C. Martín Villalba, A. Urquía Moraleda, y M. Á. Rubio González, *Lenguajes de programación*. UNED - Universidad Nacional de Educación a Distancia, 2021. Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://0x102720b-y-https-elibro-net.itmsp.museknowledge.com/es/ereader/uta/184827>
- [2] «ENIAC | History, Computer, Stands For, Machine, & Facts | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/ENIAC>
- [3] «Plankalkül | computer language | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/Plankalkul>
- [4] «Computer programming language | Definition, Examples, & Popularity | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/computer-programming-language>
- [5] «Fortran | IBM». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.ibm.com/history/fortran>
- [6] U. S. Team, «What Is Lisp And Why Did It Shape AI?», UPI Study. Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.upistudy.com/blog/artificial-intelligence/what-is-lisp-and-why-did-it-shape-ai>
- [7] «COBOL | Definition & Facts | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/COBOL>
- [8] «ALGOL 60 | computer language | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/ALGOL-60>



- [9] «ALGOL | Programming, Syntax & Compiler | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/ALGOL-computer-language>
- [10] «Computer programming language | Definition, Examples, & Popularity | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/computer-programming-language>
- [11] «Chistory». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.nokia.com/bell-labs/about/dennis-m-ritchie/chist.html>
- [12] «38 programming languages. Tried them all!», DEV Community. Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://dev.to/johnrushx/38-programming-languages-which-is-best-584f>
- [13] «Perl | Definition, History, & Facts | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/Perl>
- [14] «Python | Definition, Language, History, & Facts | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/Python-computer-language>
- [15] «Java | Definition & Facts | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/Java-computer-programming-language>
- [16] «ENIAC | Historia, Computadora, Siglas, Máquina y Datos | Britannica». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.britannica.com/technology/ENIAC>
- [17] «38 programming languages. Tried them all!», DEV Community. Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://dev.to/johnrushx/38-programming-languages-which-is-best-584f>
- [18] «Rust Programming Language». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://rust-lang.org/>
- [19] B. K. Ward, «Microsoft Unveils Javascript Tool TypeScript ->», ADTmag. Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://adtmag.com/articles/2012/10/02/microsoft-javascript-tool.aspx>
- [20] «WWDC: Apple launches new “Swift” programming language for Mac and iOS – GeekWire». Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://www.geekwire.com/2014/wwdc-apple-launches-new-swift-programming-language-mac-ios/>
- [21] N. Friedman, «Introducing GitHub Copilot: your AI pair programmer», The GitHub Blog. Accedido: 23 de agosto de 2026. [En línea]. Disponible en: <https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/>

2.12 Anexos





LA HISTORIA Y EVOLUCION DE LOS LENGUAJES DE PROGRAMACION

El progresivo desarrollo y abaratamiento del hardware de computación ha tenido una influencia fundamental en el desarrollo de los lenguajes de programación. Por una parte, el impresionante avance en las capacidades de computación ha permitido desarrollar programas cada vez más complejos, surgiendo la necesidad de desarrollar metodologías de programación adecuadas para el desarrollo de programas de grandes dimensiones y complejidad, y lenguajes de programación que soporten dichos desarrollos.[1]

1. La Era del Silicio Primitivo (Antes de FORTRAN, 1943–1950)

Año	Lenguaje / Hito	Innovación clave
1943	ENIAC – Programación por hardware	Se programaba físicamente conectando cables; no existía código digital.[2]
1945	Plankalkül – Konrad Zuse	Primer diseño teórico con variables y condicionales independientes del hardware.[3]
1947	Código máquina puro	Primer almacenamiento de instrucciones binarias en memoria RAM.[4]

2. La Consolidación del Alto Nivel (1957–1987)

Año	Lenguaje	Autor(es)	Aporte principal
1957	FORTRAN	John Backus (IBM)	Primer lenguaje de alto nivel comercial masivo. Permitió a científicos e ingenieros escribir fórmulas matemáticas en un estilo cercano al inglés, liberando a la programación del ensamblador. Fue pionero en compiladores optimizadores de calidad.[5]
1958	LISP	John McCarthy (MIT)	Introdujo la programación funcional y la recolección automática de basura. Se convirtió en el lenguaje base para la investigación en Inteligencia Artificial, gracias a su capacidad de manipular listas y símbolos.[6]
1959	COBOL	Grace Hopper y CODASYL	Lenguaje orientado a negocios con sintaxis en inglés formal. Facilitó la programación de sistemas administrativos y financieros, convirtiéndose en estándar en bancos y empresas.[7]
1960	ALGOL 60	Comité internacional (Peter Naur, etc.)	Introdujo la estructura de bloques begin/end y la descripción formal de algoritmos. Fue el ancestro sintáctico de lenguajes como Pascal, C y Java.[8]
1964	BASIC	John Kemeny y Thomas Kurtz (Dartmouth)	Diseñado para democratizar la programación en universidades. Permitió que estudiantes de cualquier carrera pudieran



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



			programar en terminales de manera sencilla.[9]
1970	Pascal	Niklaus Wirth	Lenguaje académico que prohibió el uso de GOTO para evitar saltos caóticos. Promovió la programación estructurada y se convirtió en estándar en la enseñanza.[10]
1972	C, Smalltalk	Dennis Ritchie (Bell Labs) Alan Kay, Adele Goldberg, Dan Ingalls	Logró el balance entre control del hardware y portabilidad. Fue usado para reescribir Unix, convirtiéndose en la base de sistemas operativos y lenguajes posteriores. Padre de la Programación Orientada a Objetos pura. Introdujo el concepto de “objetos que se comunican mediante mensajes”, influyendo en Java y otros lenguajes modernos.[11]
1983	C++	Bjarne Stroustrup	Extensión de C que añadió clases y orientación a objetos. Permitió combinar rendimiento nativo con abstracción, influyendo en software de alto rendimiento.[12]
1987	Perl	Larry Wall	Lenguaje de scripting para procesamiento de texto y administración de servidores. Fue clave en los primeros sistemas de red y páginas web dinámicas.[13]

3. La Era de la Red y Sistemas Concurrentes (1991–Presente)

Año	Lenguaje	Autor(es)	Aporte principal
1991	Python	Guido van Rossum (CWI)	Introdujo la indentación obligatoria y un enfoque en la legibilidad humana. Se convirtió en el lenguaje preferido para ciencia de datos, IA y automatización.[14]
1995	Java, JavaScript, PHP	James Gosling (Sun Microsystems) Brendan Eich (Netscape) Rasmus Lerdorf	Creó la Máquina Virtual de Java (JVM), que permitió ejecutar código en cualquier hardware sin recompilar. Fue clave en aplicaciones empresariales y móviles. Lenguaje dinámico para navegadores web. Se transformó en estándar global para la web interactiva y más tarde para servidores con Node.js. Introdujo scripts incrustados en HTML, dando origen a la web dinámica y a plataformas como WordPress..[15]
2000	C#	Anders Hejlsberg (Microsoft)	Lenguaje robusto dentro del ecosistema .NET. Integró



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



			orientación a objetos y productividad empresarial.[16]
2009	Go	Robert Griesemer, Rob Pike, Ken Thompson (Google)	Rediseñó la concurrencia a gran escala con goroutines. Se convirtió en lenguaje clave para servicios en la nube y microservicios.[17]
2010	Rust	Graydon Hoare (Mozilla)	Introdujo un sistema de propiedad y tipos que garantiza seguridad de memoria sin recolector de basura. Es usado en sistemas críticos y navegadores.[18]
2012	TypeScript	Anders Hejlsberg (Microsoft)	Añadió tipado estático sobre JavaScript, permitiendo proyectos web corporativos a gran escala con mayor robustez.[19]
2014	Swift	Chris Lattner (Apple)	Reemplazó Objective-C con una sintaxis limpia y segura. Se convirtió en el lenguaje principal para desarrollo en iOS y macOS.[20]
2021– Presente	Lenguaje Natural / AI Co-Pilots	GitHub (Microsoft) y OpenA	Marcó el inicio de la programación asistida por Inteligencia Artificial . Permite que los humanos describan la lógica en lenguaje natural y que la IA genere el código automáticamente. Este paradigma cambió la forma de programar, pasando de escribir sintaxis a expresar intenciones . El anuncio histórico se realizó en el GitHub Blog en 2021 con la presentación de GitHub Copilot.[21]

Relación de Antecedentes y Lenguajes Actuales.

- FORTRAN marcó el inicio de los lenguajes de alto nivel orientados al cálculo científico. Su legado se mantiene en herramientas modernas como Python, que buscan simplificar la programación en áreas de datos y simulación.
- COBOL abrió el camino en el ámbito empresarial, al enfocarse en la gestión administrativa y financiera. Su influencia se refleja en lenguajes posteriores usados en sistemas corporativos, como Java y C#.
- ALGOL aportó la estructura formal de los algoritmos y el uso de bloques, lo que inspiró directamente a Pascal, C y más tarde a C++ y Java. Su impacto fue decisivo en la evolución de la sintaxis moderna.
- C se convirtió en la base de múltiples lenguajes gracias a su eficiencia y portabilidad. De él derivaron C++, Java, C#, Go y Rust, además de ser el núcleo sobre el cual se construyó Unix.
- C++ añadió la orientación a objetos sobre la potencia de C, influyendo en lenguajes que combinan rendimiento con abstracción, como Java y Rust.



- Smalltalk consolidó la programación orientada a objetos mediante el intercambio de mensajes entre entidades, y su influencia se percibe en lenguajes como Java, C# y Swift.
- LISP y Prolog introdujeron paradigmas distintos: el funcional y el lógico. Estos conceptos fueron fundamentales en el desarrollo de la Inteligencia Artificial y hoy se consideran antecedentes de Python y de los lenguajes naturales asistidos por IA, como GitHub Copilot.

Análisis de lenguajes contemporáneos.

- C++ es un lenguaje compilado y de tipado estático que combina programación estructurada, orientada a objetos y genérica. Su principal fortaleza es el control directo de los recursos y el alto rendimiento, lo que lo hace esencial en videojuegos, motores gráficos, sistemas embebidos y software que requiere eficiencia máxima. Aunque más complejo que otros lenguajes, sigue siendo clave en proyectos donde la optimización es crítica. [12]
- Python se ha convertido en uno de los lenguajes más populares por su facilidad de aprendizaje y su sintaxis clara. Además de ser interpretado y de tipado dinámico, cuenta con un ecosistema enorme de librerías que lo hacen versátil en campos como inteligencia artificial, ciencia de datos, automatización de procesos y desarrollo web. Su filosofía de “código legible” lo convierte en una herramienta ideal para proyectos que requieren rapidez y colaboración. [14]
- Java es un lenguaje robusto y de tipado fuerte que se ejecuta sobre la Máquina Virtual de Java (JVM), lo que le otorga portabilidad entre diferentes plataformas. Su orientación a objetos y su capacidad para manejar concurrencia lo han consolidado como un estándar en aplicaciones empresariales, sistemas de gran escala y desarrollo móvil, especialmente en Android. La estabilidad y el soporte de una comunidad amplia lo mantienen vigente. [15]
- JavaScript nació como el lenguaje de la web, pero su evolución lo ha llevado más allá del navegador. Gracias a entornos como Node.js, hoy también se utiliza en servidores, lo que lo convierte en una opción completa para aplicaciones de frontend y backend. Es dinámico y flexible, capaz de combinar estilos imperativos, funcionales y orientados a objetos basados en prototipos. Su papel como lenguaje universal de la interacción web lo hace indispensable en el desarrollo moderno. [15]
- Go destaca por su simplicidad y rapidez en la compilación. Introduce mecanismos de concurrencia fáciles de manejar, como goroutines y canales, que lo convierten en una herramienta poderosa para servicios en la nube, redes y microservicios. Su diseño busca reducir la complejidad y aumentar la productividad en proyectos de infraestructura y DevOps. [17]
- Rust se diferencia por su sistema de propiedad y tipos, que garantiza seguridad de memoria y evita errores de concurrencia antes de ejecutar el programa. Es un lenguaje pensado para sistemas críticos, navegadores y aplicaciones donde la confiabilidad es



prioritaria. Al combinar paradigmas imperativos y funcionales, ofrece potencia sin sacrificar seguridad, lo que lo convierte en una alternativa moderna frente a C y C++.[18]

Comparación de lenguajes contemporáneos

Grupo de lenguajes	de	Características principales	Ejemplos	Enfoque
Accesibles y productivos	y	Sintaxis sencilla, portabilidad, rapidez de desarrollo, gran comunidad	Python, Java, JavaScript	Facilitar el aprendizaje, acelerar proyectos, dar soporte a aplicaciones web y móviles
De rendimiento y seguridad	alto y	Control de recursos, concurrencia eficiente, seguridad de memoria, compilación rápida	C++, Go, Rust	Optimizar sistemas críticos, videojuegos, motores gráficos, servicios en la nube